

Sector de negocio

Atención al cliente / Marketing / Operaciones — empresas que recopilan opiniones de clientes (reseñas, comentarios en redes sociales, encuestas de satisfacción) y quieren entender rápidamente si el sentimiento es positivo, neutro o negativo.

Descripción del proyecto

Crear una API simple que recibe textos (comentarios, reseñas o tweets), aplica un modelo de Data Science para clasificar el sentimiento (Atrasado / Puntual → en este caso: Positivo / Neutro / Negativo o binario Positivo / Negativo) y devuelve el resultado en formato JSON, permitiendo que las aplicaciones consuman esta predicción automáticamente.

Necesidad del cliente (explicación no técnica)

Un cliente (empresa) recibe muchos comentarios y no puede leerlos todos manualmente. Quiere: saber rápidamente si los clientes se están quejando o elogiando;

priorizar respuestas a comentarios negativos;

medir la satisfacción a lo largo del tiempo.

Este proyecto ofrece una solución automática para clasificar mensajes y generar información accionable.

Validación de mercado

Analizar si el sentimiento es útil para:

acelerar la atención al cliente (identificar urgencias);

monitorear campañas de marketing;

comparar la imagen de la marca a lo largo del tiempo.

Incluso una solución simple (modelo básico) tiene valor: las pequeñas y medianas empresas utilizan herramientas similares para entender los feedbacks sin un equipo dedicado.

Expectativa para este hackathon

Público: estudiantes sin experiencia profesional en el área de tecnología, que estudiaron Back-end (Java, Spring, REST, persistencia) y Data Science (Python, Pandas, scikit-learn, notebooks).

Objetivo: entregar un MVP funcional que demuestre la integración entre DS y Back-end: un notebook con el modelo + una API que carga ese modelo y responde a las peticiones.

Alcance recomendado: clasificación binaria (Positivo / Negativo) o ternaria (Positivo / Neutro / Negativo) con un modelo simple — por ejemplo, usar TF-IDF (una técnica que transforma el texto en números, mostrando qué palabras son más importantes) junto con Regresión Logística (un modelo de aprendizaje automático que aprende a diferenciar sentimientos).

Entregables deseados

Notebook (Jupyter/Colab) del equipo de Data Science que contenga:

Exploración y limpieza de los datos (EDA);

Transformación de los textos en números con TF-IDF;

Entrenamiento de modelo supervisado (ej.: Logistic Regression, Naive Bayes);

Métricas de desempeño (Accuracy, Precision, Recall, F1-score);

Serialización del modelo (joblib/pickle).

Aplicación Back-End (preferiblemente Spring Boot en Java):

API que consume el modelo (directamente o llamando al microservicio DS) y expone el endpoint /sentiment;

Endpoint que recibe información y devuelve la predicción del modelo;

Logs y manejo de errores.

Documentación mínima (README):

Cómo ejecutar el modelo y la API;

Ejemplos de petición y respuesta (JSON);

Dependencias y versiones de las herramientas.

Demonstración funcional (Presentación corta):

Mostrar la API en acción (a través de Postman, cURL o interfaz simple);

Explicar cómo el modelo llega a la predicción.

Funcionalidades exigidas (MVP)

El servicio debe exponer un endpoint que devuelve la clasificación del sentimiento y la probabilidad asociada a esa clasificación. Ejemplo: POST /sentiment — acepta JSON con campo text y devuelve: { "prevision": "Positivo", "probabilidad": 0.87 }

Modelo entrenado y cargable: el back-end debe poder usar el modelo (cargando archivo) o hacer una petición a un microservicio DS que implemente la predicción.

Validación de input: verificar si text existe y tiene longitud mínima; devolver error amigable en caso contrario.

Respuesta clara: label (+ probabilidad en 0–1) y mensaje de error cuando sea aplicable.

Ejemplos de uso: Postman/cURL con 3 ejemplos reales (positivo, neutro, negativo).

README explicando cómo ejecutar (pasos simples) y cómo probar el endpoint.

Funcionalidades opcionales

Endpoint GET /stats con estadísticas simples (porcentaje de positivos/negativos en los últimos X comentarios).

Persistencia: guardar peticiones y predicciones en base de datos (H2 o Postgres) para análisis posteriores.

Explicabilidad básica: devolver las palabras más influyentes en la predicción (ej.: "top features": ["excelente", "servicio"]).

Interfaz simple (Streamlit / página web) para probar texto libremente.

Batch processing: endpoint para enviar varios textos en CSV y recibir predicciones en lote.

Versión multilingüe (Portugués + Español) u opción para cambiar el threshold de probabilidad.

Contenerización con Docker y docker-compose para levantar DS + Back-end juntos.

Pruebas automatizadas: algunas pruebas unitarias y una prueba de integración simple.

Orientaciones técnicas para estudiantes

Recomendamos tener cuidado al utilizar las instancias limitadas proporcionadas por los servicios always free de OCI, para no incurrir en gastos adicionales.

Equipo de Data Science

Cada equipo debe elegir o armar su propio conjunto de datos de comentarios, reseñas o publicaciones que puedan utilizarse para el análisis de sentimientos (ej.: reviews públicos, tweets, evaluaciones de productos, etc.).

usar Python, Pandas para leer/limpiar datos;

crear un modelo simple (TF-IDF + LogisticRegression de scikit-learn);

guardar el pipeline y el modelo con joblib.dump.

Poner todo en un notebook bien comentado.

Equipo de Back-End

crear una API REST (en Java con Spring Boot).

Implementar un endpoint (ej: /sentiment) que recibe la evaluación y devuelve el sentimiento

Integrar el modelo de Data Science:

vía microservicio Python (FastAPI/Flask), o

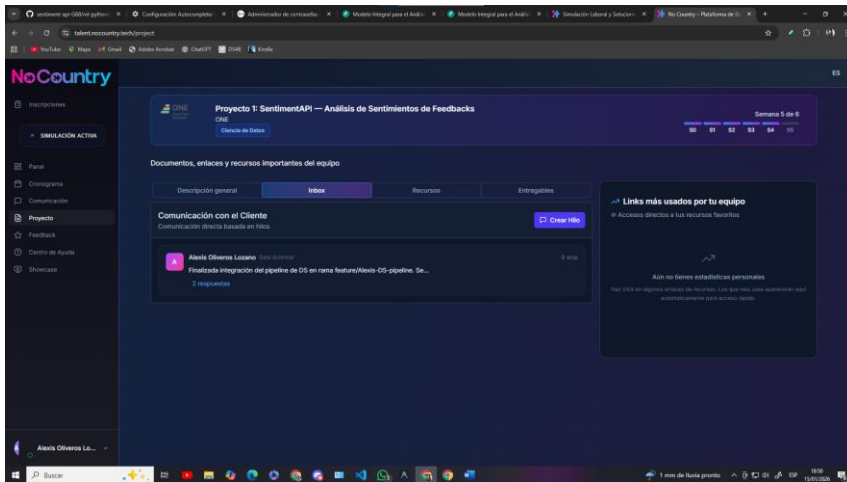
cargando el modelo exportado (ONNX, para equipos Java avanzados).

Validar entradas y devolver respuestas JSON consistentes.

Contrato de integración (definido entre DS y BE)

Recomendamos definir desde el principio el formato JSON de entrada y salida. Sigue un ejemplo:

```
{ "text": "... " } →  
{  
  "prevision": "Positivo",  
  "probabilidad": 0.9  
}
```





Proyecto 1: SentimentAPI — Análisis de Sentimientos de Feedbacks

ONE

Ciencia de Datos

Documentos, enlaces y recursos importantes del equipo

Descripción general

Inbox

Recursos

Entregables

 Lorena Raygoza

 15/1/2026

 Editado 15/1/2026



Resumen Daily 15/01/2026 : SentimentAPI G-68

Asistentes

- Fernando Falla
- Florentino Lopez

 Ver más

 Florentino Lopez

 12/1/2026

 Editado 12/1/2026



Resumen: SentimentAPI (H12-25-L-68) — 12/01/26 Estatus: Avances del funcionamiento del ML

Logros Clave Datascience: ML estable.

DS: Presento su ML funcionando y listo para la prueba piloto (MVP funcional)

 Ver más

Mensaje para NoCountry (Enfoque en Hitos y Metodología)

 Reporte de Hito DS - Equipo G68 (Sentiment API G68)

Actividad: Conclusión de Benchmarking y Estabilización de MVP de Sentimientos. Estado: Pull Request a la rama de Desarrollo (DEV).

 Ver más

 Florentino Lopez

 5/1/2026

 Editado 5/1/2026



Resumen: SentimentAPI (H12-25-L-68) — 05/01/26 Estatus: Avances Relevantes y Acuerdos Principales

Logros Clave Datascience: Compromiso de entregar el ML estable.

Backend: Integración del ML estable en la API para comenzar prueba Piloto.

Integración: Realizará la conexión de Frontend con la API para brindar en forma estable la prueba Piloto.

 Ver más

 Alexis Oliveros Lozano

 2/1/2026

 Editado 2/1/2026

 Resumen: SentimentAPI (H12-25-L-68) — 02/01/26

Estatus: Integración y Validación Técnica

 Logros Clave

Backend: Contrato de interfaz validado y DTOs implementados.

 Ver más

Comunicado oficial – Definición de horario para sprints

La encuesta para definir el horario de los sprints **finalizó el día de ayer (miércoles)**. El periodo de votación estuvo **abierto durante 3 días**, desde el **lunes hasta el miércoles**, tiempo durante el cual **todos los integrantes del equipo tuvieron la posibilidad de participar**.

Ver más